

Building & Training a Modern LLM

What Changed Since 2017 — and Why

The Classic Transformer

Token Embedding + Sinusoidal Position

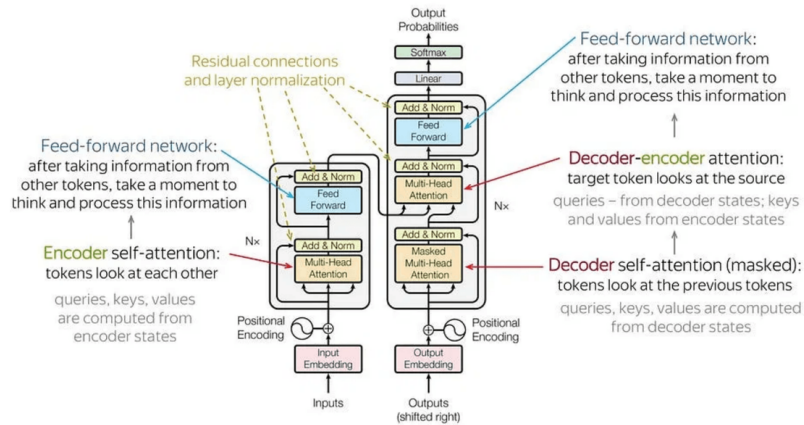
LayerNorm (Post-Norm)

Multi-Head Attention

FFN with ReLU

Essentially GPT-2 (2019)

How Transformers Work: A Step-by-Step Breakdown



Input Sequence

A sequence of words or tokens that the model will process.

The Four Upgrades

Component

Classic (2017-2019)

Modern (2023-2025)

Normalization

LayerNorm

RMSNorm

Feed-Forward

ReLU FFN

SwiGLU

Position Encoding

Sinusoidal

RoPE

Attention

Multi-Head (MHA)

Grouped Query (GQA)

Used by: Llama, Mistral, Gemma, DeepSeek, and virtually every modern open-weight LLM.

RMSNorm

LayerNorm: Mean + Variance + Normalize + Scale + Shift

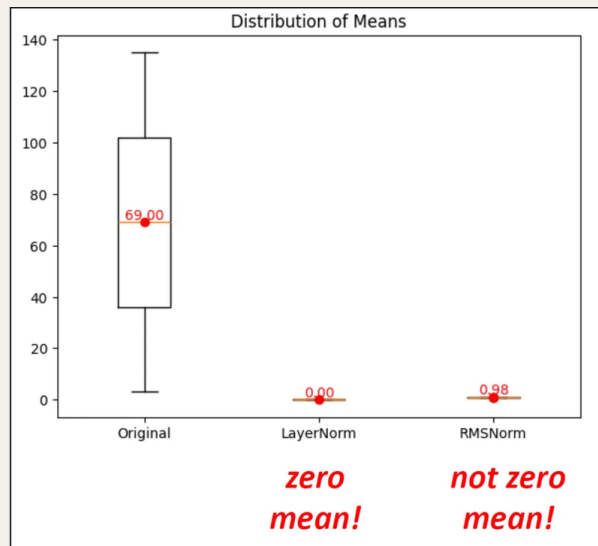
RMSNorm: RMS + Normalize + Scale

What's Gone?

Mean subtraction and the bias term (shift). Centering wasn't earning its keep.

The Benefit

Simpler, faster, and stabilizes training just as well. Crucial for scaling to billions of parameters.



RMSNorm maintains a non-zero mean, simplifying the computation without loss in stability.

RMSNorm

The Code

Simplicity in implementation

Efficiency

Fewer operations than
LayerNorm

Parameters

No mean subtraction or bias
term

Stability

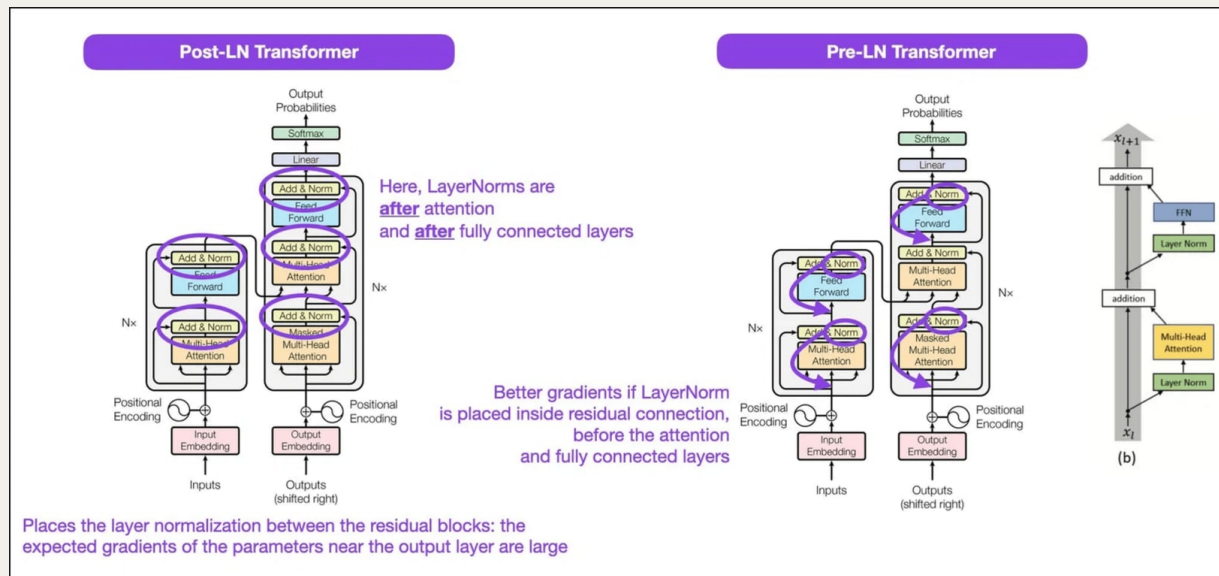
Maintains training stabilization

```
class RMSNorm(nn.Module):  
    def __init__(self, dim, eps=1e-6):  
        super().__init__()  
        self.eps = eps  
        self.weight = nn.Parameter(torch.ones(dim))  
    def forward(self, x): # Square, Mean, Root  
        rms = torch.sqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)  
        return (x / rms) * self.weight
```

Pre-Norm vs Post-Norm

- **Original:** Normalize *after* the sublayer (Post-Norm).
- **Modern:** Normalize *before* the sublayer (Pre-Norm).
- **Stability:**
Essential for deep models with 30+ layers.

Residual connection carries the raw signal.



SwiGLU (SiLU = $x \cdot \text{sigmoid}(x)$)

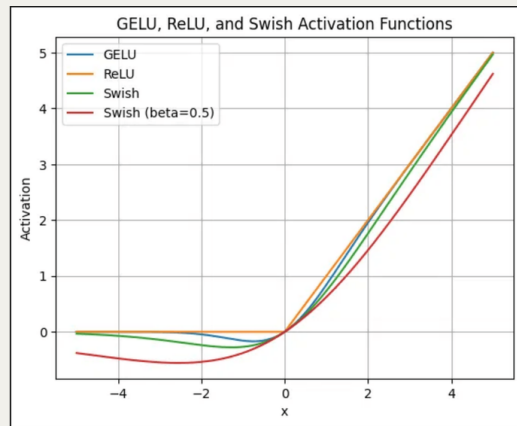
A smarter version of the Feed-Forward Network (FFN) that uses **learned gating** to control information flow.

Two Paths

One path creates a gate using SiLU; the other carries the raw signal.

SiLU Activation

$\text{SiLU}(x) = x \cdot \sigma(x)$. A smooth, non-monotonic alternative to ReLU's hard cutoff.



Comparison: ReLU vs. GELU vs. Swish (SiLU)

The Problem with Sinusoidal Positions

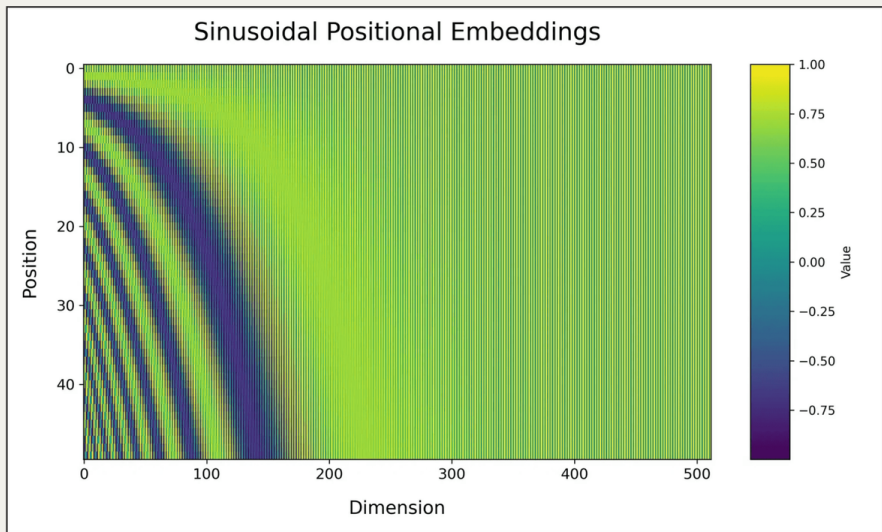
Why the 2017 approach hit a wall

2. Absolute Encoding

Encodes "I am token #47" rather than context-aware relationships.

3. No Relative Distance

Doesn't naturally capture how far apart two tokens are in the sequence.



Classic Sinusoidal Positional Encoding Pattern

RoPE

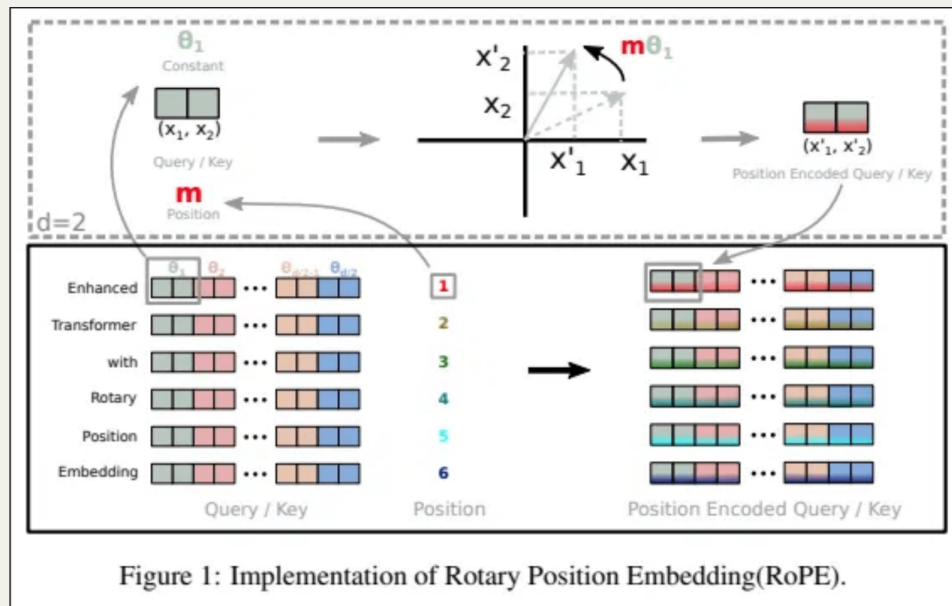
Instead of adding position to embeddings, **rotate** the query and key vectors based on their position in the sequence.

Relative Distance

The dot product between rotated vectors depends only on their relative distance, not absolute positions.

The Clock Analogy

Like hands on a clock: the angle between 3 and 5 is the same as between 7 and 9. The gap is what matters.



Visualizing vector rotation in RoPE

RoPE Mechanics

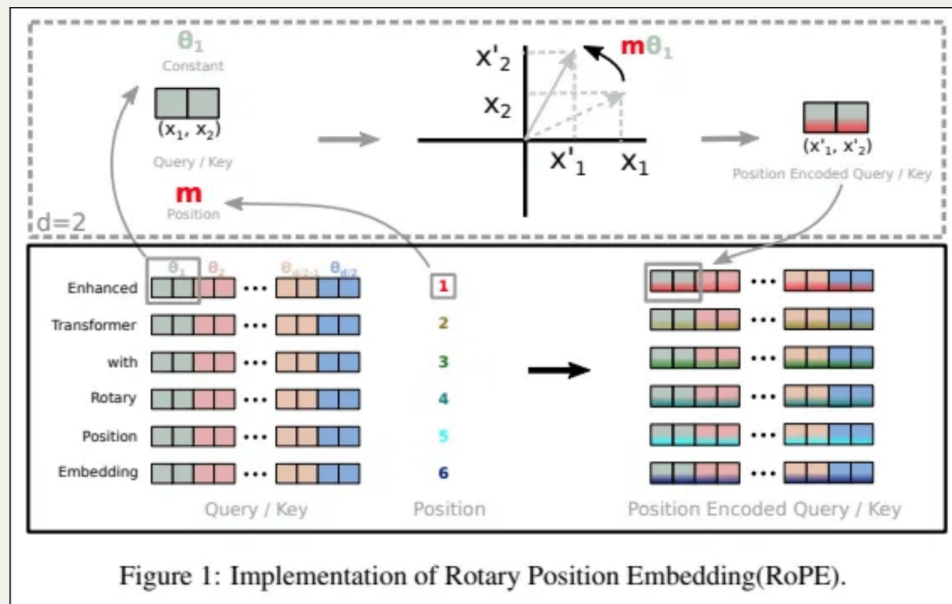
For each pair of dimensions $(2i, 2i+1)$, apply a **2D rotation** by angle θ_m .

Dimension Pairing

Dimensions are paired and rotated together based on the token's position.

Frequency Scaling

Low dims rotate fast (nearby relationships); high dims rotate slowly (distant relationships).



The Memory Bottleneck

Why standard attention fails at scale

The KV Cache

During generation, we store all past Key and Value vectors to avoid recomputation.

Linear Scaling

Memory usage grows linearly with sequence length and number of heads.

The Wall

For long contexts, the cache size exceeds GPU memory before the model even starts "thinking".

The Math (32-layer model)

$2 \times 32 \text{ heads} \times 8192 \text{ tokens}$

$\times 128 \text{ head_dim} \times 2 \text{ bytes}$

$\approx 128 \text{ MB per layer}$

~4 GB TOTAL CACHE

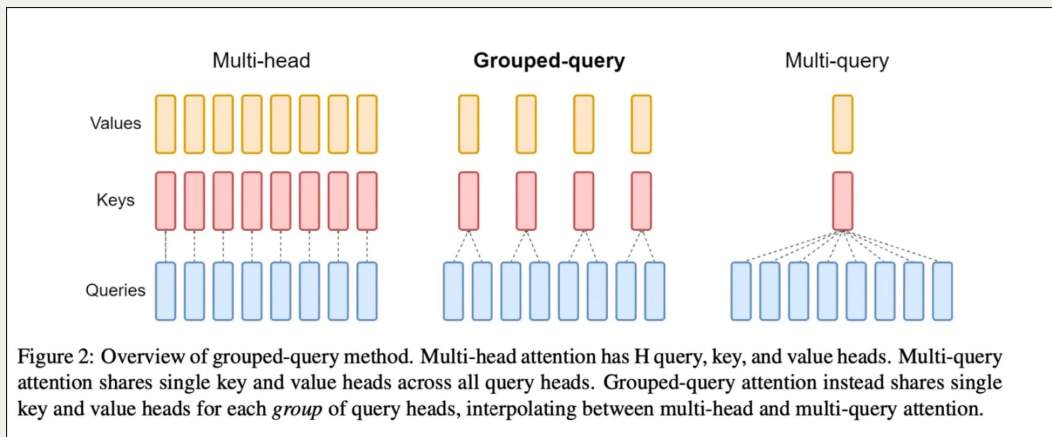
**Just for 8K tokens. 128K would require 64 GB.*

GQA

Instead of every query head having its own key and value, **Grouped Query Attention** shares KV pairs across groups of query heads.

THE SWEET SPOT

4× less KV cache memory than standard MHA, with almost no drop in model quality.



The Modern Transformer Block

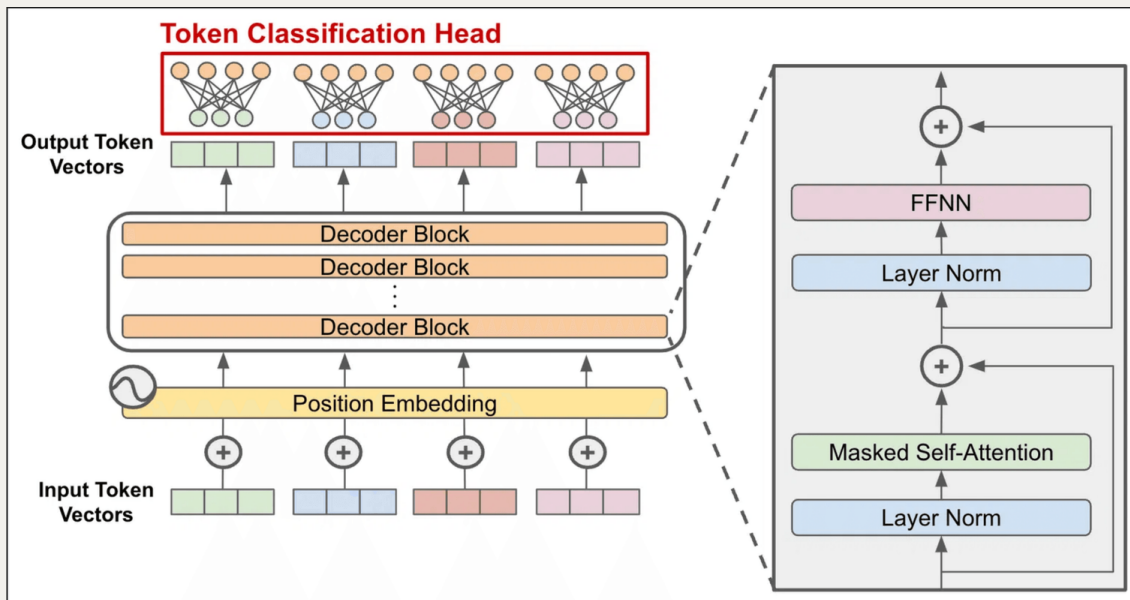
Integrating the 2024 standards

RoPE

Rotary embeddings for infinite relative context.

GQA

KV-cache sharing for memory-efficient inference.



RMSNorm

Pre-normalization for extreme training stability.

SwiGLU

Gated feed-forward paths for richer representation.

The Comparison

Classic vs. Modern Transformer

Feature	Classic (2017)	Modern (2024)
Normalization	LayerNorm (Post-Norm)	RMSNorm (Pre-Norm)
Position	Absolute Sinusoidal	Rotary (RoPE)
Attention	Multi-Head (MHA)	Grouped-Query (GQA)
Activation	ReLU	SwiGLU
Focus	Correctness	Stability & Efficiency